

尚硅谷大数据项目之电商数仓（用户行为采集平台）

（作者：尚硅谷研究院）

版本：V5.0

第1章 数据仓库概念

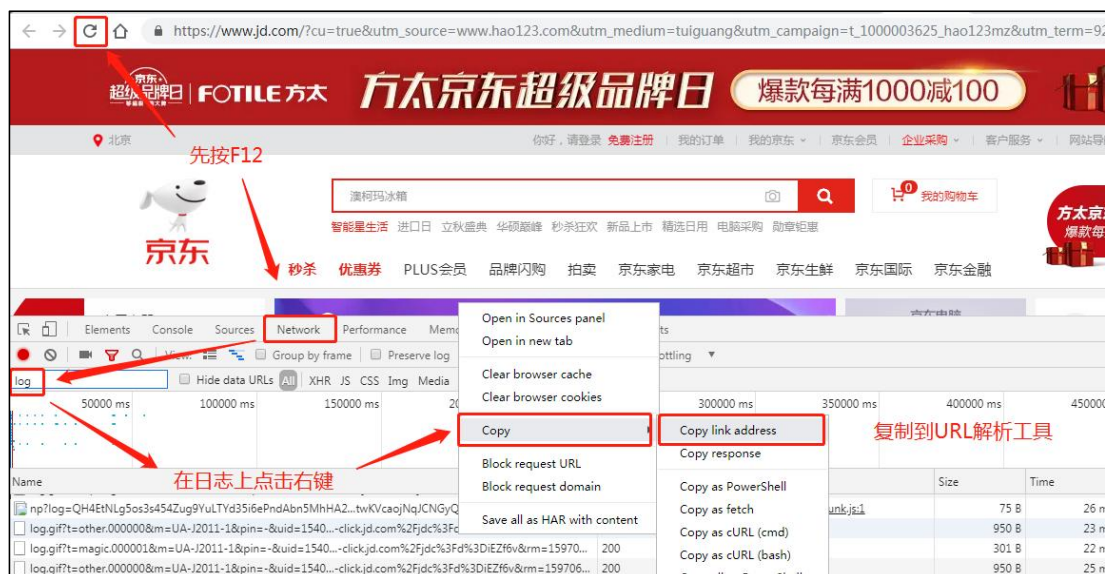
数据仓库（Data Warehouse），是为企业制定决策，提供数据支持的。可以帮助企业，改进业务流程、提高产品质量等。

数据仓库的输入数据通常包括：业务数据、用户行为数据和爬虫数据等

业务数据：就是各行业在处理事务过程中产生的数据。比如用户在电商网站中登录、下单、支付等过程中，需要和网站后台数据库进行增删改查交互，产生的数据就是业务数据。业务数据通常存储在 MySQL、Oracle 等数据库中。



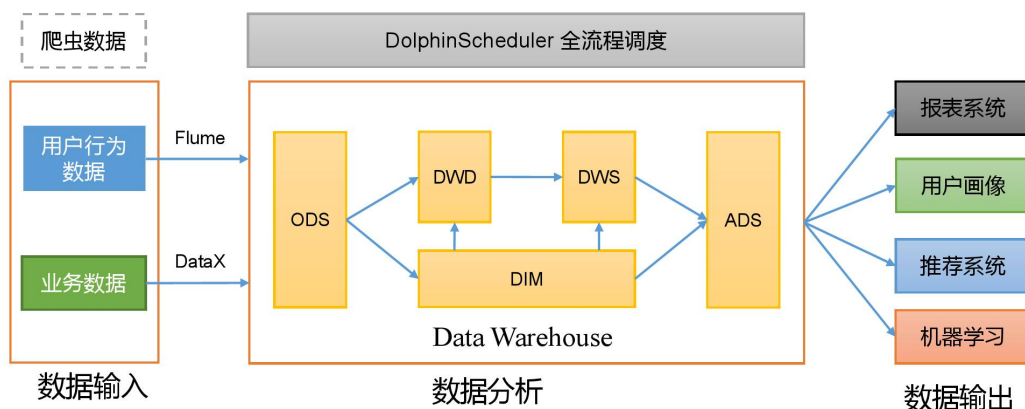
用户行为数据：用户在使用产品过程中，通过埋点收集与客户端产品交互过程中产生的数据，并发往日志服务器进行保存。比如页面浏览、点击、停留、评论、点赞、收藏等。用户行为数据通常存储在日志文件中。



爬虫数据： 通常是通过技术手段获取其他公司网站的数据。不建议同学们这样做。

数据仓库

数据仓库（Data Warehouse），是为**企业制定决策，提供数据支持**的。可以帮助企业，**改进业务流程、提高产品质量**等。



数据仓库，并不是数据的最终目的地，而是为数据最终的目的做好准备。这些准备包括对数据的：**备份、清洗、聚合、统计**等。

让天下没有难学的技术

第 2 章 项目需求及架构设计

2.1 项目需求分析

1) 采集平台

- (1) 用户行为数据采集平台搭建
- (2) 业务数据采集平台搭建

2) 离线需求

更多 **Java - 大数据 - 前端 - python** 人工智能资料下载，可百度访问：**尚硅谷官网**



电商离线指标体系.
xlsx

3) 实时需求



电商实时指标体系.
xlsx

4) 思考题

- (1) 项目技术如何选型？
- (2) 框架版本如何选型（Apache、CDH、HDP）
- (3) 服务器使用物理机还是云主机？
- (4) 如何确认集群规模？（假设每台服务器 16T 硬盘）

2.2 项目框架

2.2.1 技术选型



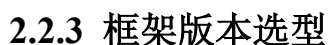
技术选型主要考虑因素：数据量大小、业务需求、行业内经验、技术成熟度、开发维护成本、总成本预算

- 数据采集传输：Flume, Kafka, DataX, Maxwell, Sqoop, FlinkCDC
- 数据存储：MySQL, HDFS, HBase, Redis, MongoDB
- 数据计算：Hive, Spark, Flink, Storm, Tez
- 数据查询：Presto, Kylin, Impala, Druid, ClickHouse, Doris
- 数据可视化：Superset, Echarts, Sugar, QuickBI, DataV, FineBI, PowerBI
- 任务调度：DolphinScheduler, Azkaban, Oozie, Airflow
- 集群监控：Zabbix, Prometheus
- 元数据管理：Atlas
- 权限管理：Ranger, Sentry

■ 离线实时共用
■ 离线
■ 实时
■ 不采用

让天下没有难学的技术

系统数据流程图



框架发行版本选型

- (1) 阿里云的EMR、MaxCompute、DataWorks
- (2) 腾讯云EMR、流计算Oceanus、数据开发治理平台WeData
- (3) 华为云EMR
- (4) 亚马逊云EMR

(1) Apache框架版本

框架	旧版本	新版本
Hadoop	3.1.3	3.3.4
Zookeeper	3.5.7	3.7.1
MySQL	5.7.16	8.0.31
Hive	3.1.2	3.1.3(修改源码)
Flume	1.9.0	1.10.1
Kafka	3.0.0	3.3.1
Spark	3.0.0	3.3.1
DataX	3.0	3.0(master分支向前, 修改源码)
Superset	1.5.3	2.0.0
DolphinScheduler	1.3.9	2.0.5
Maxwell	1.29.2	1.29.2(修改源码)
Flink	1.13.0	1.17.0
Redis		6.0.8
Hbase		2.4.11
ClickHouse		20.4.5.36-2

注意事项：框架选型尽量不要选择最新的框架，选择最新框架半年前左右的稳定版。

让天下没有难学的技术

2.2.4 服务器选型

服务器选择物理机还是云主机？

1) 物理机：

- 以128G内存，20核物理CPU，40线程，8THDD和2TSSD硬盘，戴尔品牌单台报价4W出头。一般物理机寿命5年左右。
- 需要有专业的运维人员，平均一个月1万。电费也是不少的开销。

2) 云主机：

- 云主机：以阿里云为例，差不多相同配置，每年5W。
- 很多运维工作都由云主机完成，运维相对较轻松

3) 企业选择

- 金融有钱公司选择云主机（上海）。
- 中小公司、为了融资上市，选择云主机，拉到融资后买物理机。
- 有长期打算，资金比较足，选择物理机。

让天下没有难学的技术

2.2.5 集群规模

集群规模

1) 如何确认集群规模？（假设：每台服务器16T磁盘，128G内存）

（1）每天日活跃用户100万，每人一天平均100条： $100万 * 100条 = 1亿条$

（2）每条日志1K左右，每天1亿条： $100000000 / 1024 / 1024 = 约100G$

（3）1年内不扩容服务器来算： $100G * 365天 = 约36T$

（4）保存3副本： $36T * 3 = 108T$

（5）预留20%~30%Buf= $108T / 0.7 = 154T$

（6）算到这： $约16T * 10台服务器$

2) 如果考虑数仓分层？数据采用压缩？需要重新再计算

让天下没有难学的技术

2.2.6 集群资源规划设计

在企业中通常会搭建一套生产集群和一套测试集群。生产集群运行生产任务，测试集群用于上线前代码编写和测试。

1) 生产集群

（1）参考腾讯云 EMR 官方推荐部署



节点配置

高可用 ☒ 开启

节点类型	节点规格	节点数量
Master节点配置	标准型SA3: 8核16G 系统盘: SSD云盘50G*1 数据盘: SSD云盘200G*1	2
Core节点配置	标准型SA3: 4核8G 系统盘: SSD云盘50G*1 数据盘: SSD云盘200G*1	3
Task节点配置	标准型SA3: 4核8G 系统盘: SSD云盘50G*1 数据盘: SSD云盘200G*1	0
Common节点配置	标准型SA3: 2核4G 系统盘: SSD云盘50G*1 数据盘: SSD云盘200G*1	3

配置费用 11.62元/小时
11.62元/小时

上一步 下一步

- **Master 节点：**管理节点，保证集群的调度正常进行；主要部署 NameNode、ResourceManager、HMaster 等进程；非 HA 模式下数量为 1，HA 模式下数量为 2。
- **Core 节点：**为计算及存储节点，您在 HDFS 中的数据全部存储于 core 节点中，因此为了保证数据安全，扩容 core 节点后不允许缩容；主要部署 DataNode、NodeManager、RegionServer 等进程。非 HA 模式下数量 ≥ 2 ，HA 模式下数量 ≥ 3 。

更多 Java - 大数据 - 前端 - python 人工智能资料下载，可百度访问：尚硅谷官网

➤ **Common 节点:** 为 HA 集群 Master 节点提供数据共享同步以及高可用容错服务;

主要部署分布式协调器组件, 如 **ZooKeeper**、JournalNode 等节点。非 HA 模式数量为 0, HA 模式下数量≥3。

(2) 消耗内存的分开部署

(3) 数据传输数据比较紧密的放在一起 (Kafka、clickhouse)

(4) 客户端尽量放在一到两台服务器上, 方便外部访问

(5) 有依赖关系的尽量放到同一台服务器 (例如: Ds-worker 和 hive/spark)

Master	Master	core	core	core	common	common	common
nn	nn	dn	dn	dn	JournalNode	JournalNode	JournalNode
rm	rm	nm	nm	nm			
					zk	zk	zk
hive	hive	hive	hive	hive			
		kafka	kafka	kafka			
spark	spark	spark	spark	spark			
datax	datax	datax	datax	datax			
Ds-master	Ds-master	Ds-worker	Ds-worker	Ds-worker			
maxwell							
superset							
mysql							
flume	flume						
flink	flink						
		clickhouse					
			redis				
hbase							

2) 测试集群服务器规划

服务名称	子服务	服务器 hadoop102	服务器 hadoop103	服务器 hadoop104
HDFS	NameNode	√		
	DataNode	√	√	√
	SecondaryNameNode			√
Yarn	NodeManager	√	√	√
	Resourcemanager		√	
Zookeeper	Zookeeper Server	√	√	√
Flume (采集日志)	Flume	√	√	
Kafka	Kafka	√	√	√
Flume (消费 Kafka 日志)	Flume			√

Flume (消费 Kafka 业务)	Flume			√
Hive		√	√	√
MySQL	MySQL	√		
DataX		√	√	√
Spark		√	√	√
DolphinScheduler	ApiApplicationServer	√		
	AlertServer	√		
	MasterServer	√		
	WorkerServer	√	√	√
	LoggerServer	√	√	√
Superset	Superset	√		
Flink		√		
ClickHouse		√		
Redis		√		
Hbase		√		
服务数总计		20	11	12

第 3 章 用户行为日志

3.1 用户行为日志概述

用户行为日志的内容，主要包括用户的各项**行为信息**以及行为所处的**环境信息**。收集这些信息的主要目的是优化产品和为各项分析统计指标提供数据支撑。收集这些信息的手段通常为**埋点**。

目前主流的埋点方式，有**代码埋点（前端/后端）、可视化埋点、全埋点**等。

代码埋点是通过调用埋点 SDK 函数，在需要埋点的业务逻辑功能位置调用接口，上报埋点数据。例如，我们对页面中的某个按钮埋点后，当这个按钮被点击时，可以在这个按钮对应的 OnClick 函数里面调用 SDK 提供的数据发送接口，来发送数据。

可视化埋点只需要研发人员集成采集 SDK，不需要写埋点代码，业务人员就可以通过访问分析平台的“圈选”功能，来“圈”出需要对用户行为进行捕捉的控件，并对该事件进行命名。圈选完毕后，这些配置会同步到各个用户的终端上，由采集 SDK 按照圈选的配置自动进行用户行为数据的采集和发送。

全埋点是通过在产品中嵌入 SDK，前端自动采集页面上的全部用户行为事件，上报埋点数据，相当于做了一个统一的埋点。然后再通过界面配置哪些数据需要在系统里面进行分析。

3.2 用户行为日志内容

本项目收集和用户的用户行为信息主要有**页面浏览记录、动作记录、曝光记录、启动记录和错误记录。**

3.2.1 页面浏览记录

页面浏览记录，记录的是访客对页面的浏览行为，该行为的环境信息主要有用户信息、时间信息、地理位置信息、设备信息、应用信息、渠道信息及页面信息等。


页面浏览记录




用户信息	包括用户ID、设备ID
时间信息	用户跳入页面的时间
地理位置信息	用户浏览页面时所处的地理位置
设备信息	包括设备品牌、设备型号、设备系统
应用信息	指用户访问的应用信息，例如应用版本
渠道信息	指应用的下载渠道
页面信息	用户浏览的页面相关信息，包括页面ID，页面对象

让天下没有难学的技术

3.2.2 动作记录

动作记录，记录的是用户的业务操作行为，该行为的环境信息主要有用户信息、时间信息、地理位置信息、设备信息、应用信息、渠道信息 及动作目标对象信息等。



用户信息	包括用户ID、设备ID
时间信息	动作时间
地理位置信息	动作发生时所处的地理位置
设备信息	包括设备品牌、设备型号、设备系统
应用信息	指用户访问的应用信息，例如应用版本
渠道信息	指应用的下载渠道
动作目标信息	动作目标对象相关信息，包括对象类型，对象ID

让天下没有难学的技术

3.2.3 曝光记录

曝光记录，记录的是曝光行为，该行为的环境信息主要有用户信息、时间信息、地理位置信息、设备信息、应用信息、渠道信息及曝光对象信息等。



用户信息	包括用户ID、设备ID
时间信息	曝光时间
地理位置信息	曝光行为发生时所处的地理位置
设备信息	包括设备品牌、设备型号、设备系统
应用信息	指用户访问的应用信息，例如应用版本
渠道信息	指应用的下载渠道
曝光对象信息	曝光对象相关信息，包括对象类型，对象ID

让天下没有难学的技术

3.2.4 启动记录

启动记录，记录的是用户启动应用的行为，该行为的环境信息主要有用户信息、时间信息、地理位置信息、设备信息、应用信息、渠道信息、启动类型及开屏广告信息等。



用户信息	包括用户ID、设备ID
时间信息	启动时间
地理位置信息	启动时所处的地理位置
设备信息	包括设备品牌、设备型号、设备系统
应用信息	指用户访问的应用信息，例如应用版本
渠道信息	指应用的下载渠道
启动类型	包括图标和推送
开屏广告信息	包括广告ID等信息

让天下没有难学的技术

3.2.5 错误记录

启动记录，记录的是用户在使用应用过程中的报错行为，该行为的环境信息主要有用户信息、时间信息、地理位置信息、设备信息、应用信息、渠道信息、以及可能与报错相关的页面信息、动作信息、曝光信息和动作信息。

3.3 用户行为日志格式

我们的日志结构大致可分为两类，一是**页面日志**，二是**启动日志**。

3.3.1 页面日志

页面日志，以页面浏览为单位，即一个页面浏览记录，生成一条页面埋点日志。一条**完整**的页面日志包含，一个页面浏览记录，若干个用户在该页面所做的动作记录，若干个该页面的曝光记录，以及一个在该页面发生的报错记录。除上述行为信息，页面日志还包含了这些行为所处的各种环境信息，包括用户信息、时间信息、地理位置信息、设备信息、应用信息、渠道信息等。

```
{
  "common": {
    "ar": "15",
    "ba": "iPhone",
    "ch": "Appstore",
    "is_new": "1",
    "md": "iPhone 8",
    "mid": "YXfhjAYH6As2z9Iq",
    "os": "iOS 13.2.9",
    -- 环境信息
    -- 省份 ID
    -- 手机品牌
    -- 渠道
    -- 是否首日使用，首次使用的当日，该字段
    -- 手机型号
    -- 设备 id
    -- 操作系统
  }
}
```

```
"sid": "3981c171-558a-437c-be10-da6d2553c517"    -- 会话 id
"uid": "485",                                     -- 会员 id
"vc": "v2.1.134"                                  -- app 版本号
},
"actions": [{                                     -- 动作 (事件)
  "action_id": "favor_add",                       -- 动作 id
  "item": "3",                                    -- 目标 id
  "item_type": "sku_id",                          -- 目标类型
  "ts": 1585744376605                             -- 动作时间戳
},
],
"displays": [{                                   -- 曝光
  "displayType": "query",                         -- 曝光类型
  "item": "3",                                    -- 曝光对象 id
  "item_type": "sku_id",                          -- 曝光对象类型
  "order": 1,                                     -- 出现顺序
  "pos_id": 2                                     -- 曝光位置
  "pos_seq": 1                                   -- 曝光序列号 (同一坑位多个对象的编号)
},
{
  "displayType": "promotion",
  "item": "6",
  "item_type": "sku_id",
  "order": 2,
  "pos_id": 1
  "pos_seq": 1
},
{
  "displayType": "promotion",
  "item": "9",
  "item_type": "sku_id",
  "order": 3,
  "pos_id": 3
  "pos_seq": 1
},
{
  "displayType": "recommend",
  "item": "6",
  "item_type": "sku_id",
  "order": 4,
  "pos_id": 2
  "pos_seq": 1
},
{
  "displayType": "query ",
  "item": "6",
  "item_type": "sku_id",
  "order": 5,
  "pos_id": 1
  "pos_seq": 1
}
],
"page": {                                         -- 页面信息
  "during_time": 7648,                           -- 持续时间毫秒
}
```

```
{
  "item": "3",
  "item_type": "sku_id",
  "last_page_id": "login",
  "page_id": "good_detail",
  "from_pos_id": 999,
  "from_pos_seq": 999,
  "refer_id": "2",
  "sourceType": "promotion",
},
"err": {
  "error_code": "1234",
  "msg": "*****"
},
"ts": 1585744374423
}
```

3.3.2 启动日志

启动日志以启动为单位，及一次启动行为，生成一条启动日志。一条完整的启动日志包括一个启动记录，一个本次启动时的报错记录，以及启动时所处的环境信息，包括用户信息、时间信息、地理位置信息、设备信息、应用信息、渠道信息等。

```
{
  "common": {
    "ar": "370000",
    "ba": "Honor",
    "ch": "wandoujia",
    "is_new": "1",
    "md": "Honor 20s",
    "mid": "eQF5boERMJFOujcp",
    "os": "Android 11.0",
    "sid": "a1068e7a-e25b-45dc-9b9a-5a55ae83fc81",
    "uid": "76",
    "vc": "v2.1.134"
  },
  "start": {
    "entry": "icon",
    "loading_time": 18803,
    "open_ad_id": 7,
    "open_ad_ms": 3449,
    "open_ad_skip_ms": 1989
  },
  "err": {
    "error_code": "1234",
    "msg": "*****"
  },
  "ts": 1585744304000
}
```

3.4 服务器和 JDK 准备

3.4.1 服务器准备

安装如下文档配置步骤，分别安装 hadoop102、hadoop103、hadoop104 三台主机。



尚硅谷大数据技术
之hadoop（服务器

3.4.2 阿里云服务器准备（可选）



尚硅谷大数据技术
之阿里云服务器购买

3.4.3 编写集群分发脚本 xsync

1) xsync 集群分发脚本

(1) 需求：循环复制文件到所有节点的相同目录下

(2) 需求分析

① rsync 命令原始拷贝：

```
rsync -av /opt/module root@hadoop103:/opt/
```

② 期望脚本：

xsync 要同步的文件名称

③ 说明：在/home/atguigu/bin 这个目录下存放的脚本，atguigu 用户可以在系统任何地方直接执行。

```
[atguigu@hadoop102 ~]$ echo $PATH
/usr/local/bin:/usr/bin:/usr/local/sbin:/usr/sbin:/home/atguigu/.
local/bin:/home/atguigu/bin
```

(3) 脚本实现

①在用的家目录/home/atguigu 下创建 bin 文件夹

```
[atguigu@hadoop102 ~]$ mkdir bin
```

②在/home/atguigu/bin 目录下创建 xsync 文件，以便全局调用

```
[atguigu@hadoop102 ~]$ cd /home/atguigu/bin
[atguigu@hadoop102 ~]$ vim xsync
```

在该文件中编写如下代码

```
#!/bin/bash
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：尚硅谷官网

```
#1. 判断参数个数
if [ $# -lt 1 ]
then
    echo Not Enough Argument!
    exit;
fi

#2. 遍历集群所有机器
for host in hadoop102 hadoop103 hadoop104
do
    echo ===== $host =====
    #3. 遍历所有目录，挨个发送
    for file in $@
    do
        #4 判断文件是否存在
        if [ -e $file ]
        then
            #5. 获取父目录
            pdir=$(cd -P $(dirname $file); pwd)
            #6. 获取当前文件的名称
            fname=$(basename $file)
            ssh $host "mkdir -p $pdir"
            rsync -av $pdir/$fname $host:$pdir
        else
            echo $file does not exists!
        fi
    done
done
```

③修改脚本 xsync 具有执行权限

```
[atguigu@hadoop102 bin]$ chmod 777 xsync
```

④测试脚本

```
[atguigu@hadoop102 bin]$ xsync xsync
```

3.4.4 SSH 无密登录配置

说明：这里面只配置了 hadoop102、hadoop103 到其他主机的无密登录；因为 hadoop102 配置的是 NameNode，hadoop103 配置的是 ResourceManager，都要求对其他节点无密访问。

（1）hadoop102 上生成公钥和私钥：

```
[atguigu@hadoop102 .ssh]$ ssh-keygen -t rsa
```

然后敲（三个回车），就会生成两个文件 id_rsa（私钥）、id_rsa.pub（公钥）。

（2）将 hadoop102 公钥拷贝到要免密登录的目标机器上

```
[atguigu@hadoop102 .ssh]$ ssh-copy-id hadoop102
[atguigu@hadoop102 .ssh]$ ssh-copy-id hadoop103
[atguigu@hadoop102 .ssh]$ ssh-copy-id hadoop104
```

（3）hadoop103 上生成公钥和私钥：

```
[atguigu@hadoop103 .ssh]$ ssh-keygen -t rsa
```

然后敲（三个回车），就会生成两个文件 id_rsa（私钥）、id_rsa.pub（公钥）。

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：尚硅谷官网

(4) 将 hadoop103 公钥拷贝到要免密登录的目标机器上

```
[atguigu@hadoop103 .ssh]$ ssh-copy-id hadoop102
[atguigu@hadoop103 .ssh]$ ssh-copy-id hadoop103
[atguigu@hadoop103 .ssh]$ ssh-copy-id hadoop104
```

3.4.5 JDK 准备

1) 卸载现有 JDK (3 台节点)

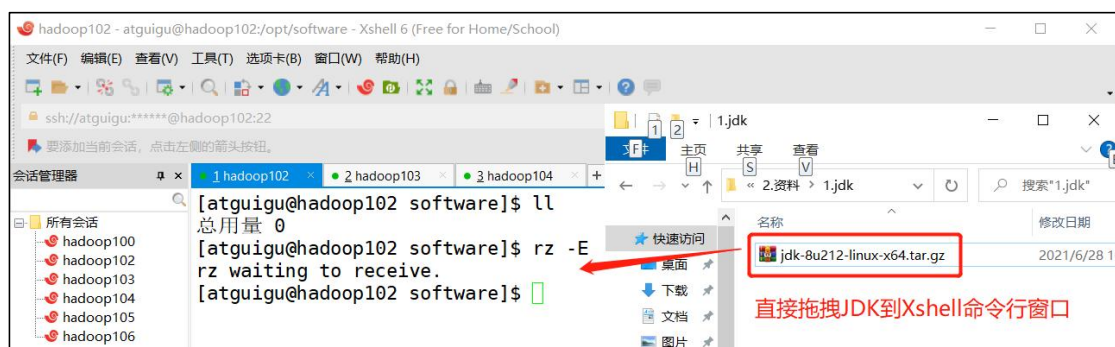
```
[atguigu@hadoop102 opt]# sudo rpm -qa | grep -i java | xargs -n1 sudo
rpm -e --nodeps

[atguigu@hadoop103 opt]# sudo rpm -qa | grep -i java | xargs -n1 sudo
rpm -e --nodeps

[atguigu@hadoop104 opt]# sudo rpm -qa | grep -i java | xargs -n1 sudo
rpm -e --nodeps
```

- (1) rpm -qa: 表示查询所有已经安装的软件包
- (2) grep -i: 表示过滤时不区分大小写
- (3) xargs -n1: 表示一次获取上次执行结果的一个值
- (4) rpm -e --nodeps: 表示卸载软件

2) 用 XShell 工具将 JDK 导入到 hadoop102 的 /opt/software 文件夹下面



3) 在 Linux 系统下的 opt 目录中查看软件包是否导入成功

```
[atguigu@hadoop102 software]# ls /opt/software/
```

看到如下结果:

```
jdk-8u212-linux-x64.tar.gz
```

4) 解压 JDK 到 /opt/module 目录下

```
[atguigu@hadoop102 software]# tar -zxvf jdk-8u212-linux-x64.tar.gz
-C /opt/module/

[atguigu@hadoop102 module]$ mv jdk1.8.0_212/ jdk-1.8.0
```

5) 配置 JDK 环境变量

(1) 新建 /etc/profile.d/my_env.sh 文件

```
[atguigu@hadoop102 module]# sudo vim /etc/profile.d/my_env.sh
```

添加如下内容, 然后保存 (:wq) 退出。

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载, 可百度访问: [尚硅谷官网](#)

```
#JAVA_HOME
export JAVA_HOME=/opt/module/jdk-1.8.0
export PATH=$PATH:$JAVA_HOME/bin
```

(2) 让环境变量生效

```
[atguigu@hadoop102 software]$ source /etc/profile.d/my_env.sh
```

6) 测试 JDK 是否安装成功

```
[atguigu@hadoop102 module]# java -version
```

如果能看见以下结果、则 Java 正常安装。

```
java version "1.8.0_212"
```

7) 分发 JDK

```
[atguigu@hadoop102 module]$ xsync /opt/module/jdk-1.8.0
```

8) 分发环境变量配置文件

```
[atguigu@hadoop102 module]$ sudo /home/atguigu/bin/xsync
/etc/profile.d/my_env.sh
```

9) 分别在 hadoop103、hadoop104 上执行 source

```
[atguigu@hadoop103 module]$ source /etc/profile.d/my_env.sh
[atguigu@hadoop104 module]$ source /etc/profile.d/my_env.sh
```

3.4.6 环境变量配置说明

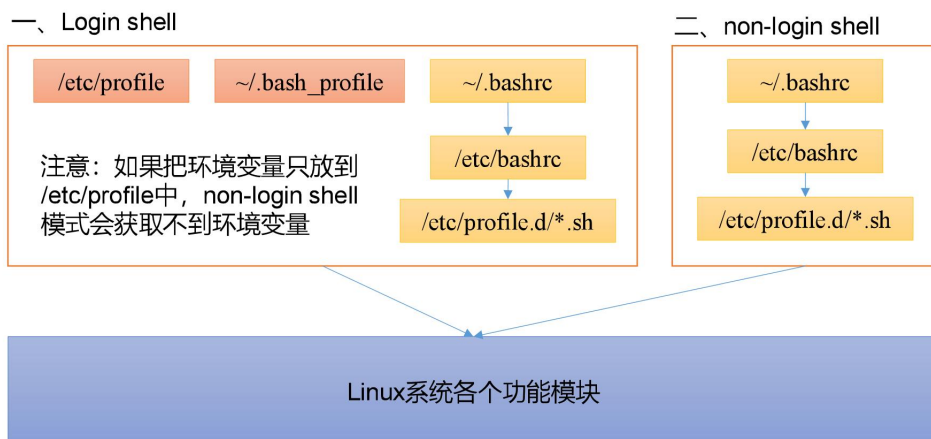
Linux 的环境变量可在多个文件中配置，如 `/etc/profile`，`/etc/profile.d/*.sh`，`~/.bashrc`，`~/.bash_profile` 等，下面说明上述几个文件之间的关系和区别。

bash 的运行模式可分为 **login shell** 和 **non-login shell**。

例如，我们通过终端，输入用户名、密码，登录系统之后，得到就是一个 login shell。而当我们执行以下命令 `ssh hadoop103 command`，在 hadoop103 执行 `command` 的就是一个 non-login shell。



登录Shell和非登录Shell区别



让天下没有难学的技术

这两种 shell 的主要区别在于，它们启动时会加载不同的配置文件，**login shell** 启动时会加载 **/etc/profile**，**~/.bash_profile**，**~/.bashrc**。**non-login shell** 启动时会加载 **~/.bashrc**。

而在加载 **~/.bashrc**（实际是 **~/.bashrc** 中加载的 **/etc/bashrc**）或 **/etc/profile** 时，都会执行如下代码片段，

```
for i in /etc/profile.d/*.sh /etc/profile.d/sh.local ; do
    if [ -r "$i" ]; then
        if [ "${-#*i}" != "$-" ]; then
            . "$i"
        else
            . "$i" >/dev/null
        fi
    fi
done
```

因此不管是 login shell 还是 non-login shell，启动时都会加载 **/etc/profile.d/*.sh** 中的环境变量。

3.5 数据模拟

3.5.1 使用说明

1) 将 **application.yml**、**gmall-remake-mock-2023-02-17.jar**、**path.json**、**logback.xml** 上传到 **hadoop102** 的 **/opt/module/applog** 目录下

(1) 创建 applog 路径

```
[atguigu@hadoop102 module]$ mkdir /opt/module/applog
```

(2) 上传文件到 **/opt/module/applog** 目录

2) 配置文件

(1) **application.yml** 文件

可以根据需求生成对应日期的用户行为日志。

```
[atguigu@hadoop102 applog]$ vim application.yml
```

修改如下内容

```
# 外部配置打开
logging.config: ./logback.xml

#http 模式下，发送的地址
mock:
  log:
    type: "file"      #"file" "http" "kafka" "none"
    http:
      url: "http://localhost:8090/applog"
    kafka:
      server: "hadoop102:9092,hadoop102:9092,hadoop102:9092"
      topic: "topic_log"
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：尚硅谷官网

```
spring:
  datasource:
    type: com.alibaba.druid.pool.DruidDataSource
    druid:
      url:
jdbc:mysql://hadoop102:3306/gmall?characterEncoding=utf-8&allowPublicKeyRetrieval=true&useSSL=false&serverTimezone=GMT%2B8
      username: root
      password: "000000"
      driver-class-name: com.mysql.cj.jdbc.Driver
      max-active: 20
      test-on-borrow: true

mybatis-plus.global-config.db-config.field-strategy: not_null
mybatis-plus:
  mapper-locations: classpath:mapper/*.xml

mybatis:
  mapper-locations: classpath:mapper/*.xml

#业务日期， 并非 Linux 系统时间的日期，而是生成模拟数据的日期
mock.date: "2022-06-08"

# 日志是否写入数据库一份 写入 z_log 表中
mock.log.db.enable: 1

# 清空
mock.clear.busi: 1

# 清空用户
mock.clear.user: 0

# 批量生成新用户
mock.new.user: 0
  #session 次数
mock.user-session.count: 200
  #设备最大值
mock.max.mid: 1000000

# 是否针对实时生成数据，若启用（置为 1）则数据的 yyyy-MM-dd 与 mock.date 一致而 HH:mm:ss 与系统时间一致；若禁用则数据的 yyyy-MM-dd 与 mock.date 一致而 HH:mm:ss 随机分布，此处禁用
mock.if-realtime: 0
#访问时间分布权重
mock.start-time-weight:
"10:5:0:0:0:0:5:5:5:10:10:15:20:10:10:10:10:10:20:25:30:35:30:20"

#支付类型占比 支付宝 : 微信 : 银联
mock.payment_type_weight: "40:50:10"

#页面平均访问时间
mock.page.during-time-ms: 20000
#错误概率 百分比
```

```
mock.error.rate: 3
    #每条日志发送延迟 ms
mock.log.sleep: 100
    #课程详情来源 用户查询, 商品推广, 智能推荐, 促销活动
mock.detail.source-type-rate: "40:25:15:20"

mock.if-cart-rate: 100

mock.if-favor-rate: 70

mock.if-order-rate: 100

mock.if-refund-rate: 50


    #搜索关键词
mock.search.keyword: "java,python, 多线程, 前端, 数据库, 大数据,hadoop,flink"


    #用户数据变化概率
mock.user.update-rate: 20


# 男女浏览品牌比重 (11 品牌)
mock.tm-weight.male: "3:2:5:5:5:1:1:1:1:1:1"
mock.tm-weight.female: "1:5:1:1:2:2:2:5:5:5:5"


# 外连类型比重 (5 种)
mock.refer-weight: "10:2:3:4:5"


# 线程池相关配置
mock.pool.core: 20
mock.pool.max-core: 100
```

(2) path.json, 该文件用来配置访问路径

根据需求, 可以灵活配置用户点击路径。

```
[
  {
    "path": ["start_app", "home", "search", "good_list", "good_detail", "good_detail", "good_detail", "cart", "order", "payment", "mine", "order_list", "end"], "rate": 100 },
    {
      "path": ["start_app", "home", "good_list", "good_detail", "good_detail", "good_detail", "cart", "end"], "rate": 30 },
      {
        "path": ["start_app", "home", "activity1111", "good_detail", "cart", "good_detail", "cart", "order", "payment", "end"], "rate": 30 },
        {
          "path": [ "activity1111", "good_detail", "activity1111", "good_detail", "order", "payment", "end"], "rate": 200 },
          ]
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载, 可百度访问: [尚硅谷官网](#)

```
{ "path": [ "start_app", "home", "activity1111", "good_detail", "order",
"payment", "end"], "rate": 200 },
{ "path": [
"start_app", "home",
"good_detail", "order", "payment", "end"], "rate": 30 },
{ "path": [ "good_detail", "order", "payment", "end"], "rate": 650 },
{ "path": [ "good_detail" ], "rate": 30 },

{ "path": [ "start_app", "home", "mine", "good_detail" ], "rate": 30 },
{ "path": [
"start_app", "home",
"good_detail", "good_detail", "good_detail", "cart", "order", "payment",
"end" ], "rate": 200 },
{ "path": [
"start_app", "home",
"search", "good_list", "good_detail", "cart", "order", "payment", "end"
], "rate": 200 }
]
```

(3) logback 配置文件

可配置日志生成路径, 修改内容如下。

```
<?xml version="1.0" encoding="UTF-8"?>
<configuration>
  <property name="LOG_HOME" value="/opt/module/applog/log" />
  <appender name="console"
class="ch.qos.logback.core.ConsoleAppender">
    <target>System.out</target>
    <encoder>
      <pattern>%msg%n</pattern>
    </encoder>
  </appender>

  <appender name="console_em"
class="ch.qos.logback.core.ConsoleAppender">
    <target>System.err</target>
    <encoder>
      <pattern>%msg%n</pattern>
    </encoder>
  </appender>

  <appender name="rollingFile"
class="ch.qos.logback.core.rolling.RollingFileAppender">
    <file>${LOG_HOME}/app.log</file>
    <rollingPolicy
class="ch.qos.logback.core.rolling.TimeBasedRollingPolicy">
<fileNamePattern>${LOG_HOME}/app.%d{yyyy-MM-dd}.log</fileNamePattern>
    </rollingPolicy>
    <encoder>
      <pattern>%msg%n</pattern>
    </encoder>
  </appender>

  <!-- 将某一个包下日志单独打印日志 -->
  <logger name="com.atguigu.mock.util.LogUtil"
    level="INFO" additivity="false">
    <appender-ref ref="rollingFile" />
  <!--
    <appender-ref ref="console" />-->
```

```

</logger>
<logger      name="com.atguigu.gmallre.mock.task.UserMockTask"
level="INFO" additivity="false" >
    <appender-ref ref="console_em" />
</logger>

<!--      <logger      name="com.alibaba.druid.pool"      level="error"
additivity="false" >-->
<!--      <appender-ref ref="console" />-->
<!--      </logger>-->

<!--      <logger      name="com.atguigu.edu2021.mock.mapper"
level="debug">-->
<!--      <appender-ref ref="console" />-->
<!--      </logger>-->

<!--      <logger      name="com.atguigu.edu2021.mock.service.impl.UserInfoServiceImpl"
level="debug">
    <appender-ref ref="console" />
</logger>-->

<root level="error" >
    <appender-ref ref="console_em" />
    <!-- <appender-ref ref="async-rollingFile" /> -->
</root>
</configuration>

```

3) 生成日志

(1) 进入到/opt/module/applog 路径，执行以下命令

```

[atguigu@hadoop102 applog]$ java -jar
gmall-remake-mock-2023-02-17.jar test 100 2022-06-08

```

- ① 增加 test 参数为测试模式，只生成用户行为数据不生成业务数据。
- ② 100 为产生的用户 session 数一个 session 默认产生 1 条启动日志和 5 条页面方法日志。
- ③ 第三个参数为日志数据的日期，测试模式下不会加载配置文件，要指定数据日期只能通过命令行传参实现。
- ④ 三个参数的顺序必须与示例保持一致
- ⑤ 第二个参数和第三个参数可以省略，如果 test 后面不填写参数，默认为 1000

(2) 在/opt/module/applog/log 目录下查看生成日志

```

[atguigu@hadoop102 log]$ ll

```

3.5.2 集群日志生成脚本

(1) 在/home/atguigu/bin 目录下创建脚本 lg.sh

```

[atguigu@hadoop102 bin]$ vim lg.sh

```

(2) 在脚本中编写如下内容

```
#!/bin/bash
echo "===== hadoop102 ====="
ssh hadoop102 "cd /opt/module/applog/; nohup java -jar
gsmall-remake-mock-2023-02-17.jar $1 $2 $3 >/dev/null 2>&1 &"
```

done 注：

①/opt/module/applog/为 jar 包及配置文件所在路径

②/dev/null 代表 Linux 的空设备文件，所有往这个文件里面写入的内容都会丢失，俗称“黑洞”。

标准输入 0：从键盘获得输入 /proc/self/fd/0

标准输出 1：输出到屏幕（即控制台） /proc/self/fd/1

错误输出 2：输出到屏幕（即控制台） /proc/self/fd/2

（3）修改脚本执行权限

```
[atguigu@hadoop102 bin]$ chmod 777 lg.sh
```

（4）将 jar 包及配置文件上传至 hadoop103 的/opt/module/applog/路径

（5）启动脚本

```
[atguigu@hadoop102 module]$ lg.sh test 100
```

（6）分别在 hadoop102、hadoop103 的/opt/module/applog/log 目录上查看生成的数据

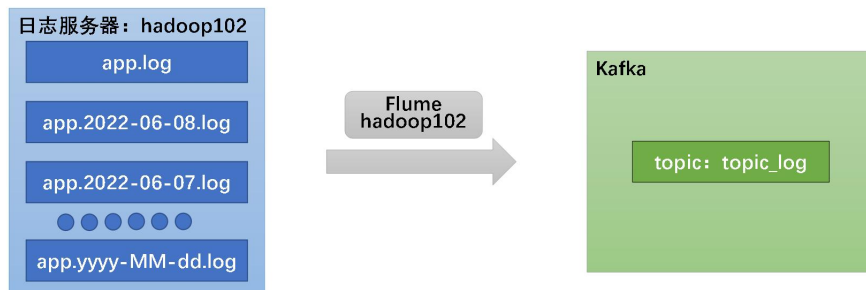
```
[atguigu@hadoop102 log]$ ls
app.log
```

第 4 章 用户行为数据采集模块

4.1 数据通道



用户行为日志数据通道



让天下没有难学的技术

4.2 环境准备

4.2.1 集群命令批量执行脚本

1) 在/home/atguigu/bin 目录下创建脚本 xcall

```
[atguigu@hadoop102 bin]$ vim xcall
```

2) 在脚本中编写如下内容

```
#!/bin/bash

for i in hadoop102 hadoop103 hadoop104
do
    echo ----- $i -----
    ssh $i "$*"
done
```

3) 修改脚本执行权限

```
[atguigu@hadoop102 bin]$ chmod 777 xcall
```

4) 启动脚本

```
[atguigu@hadoop102 bin]$ xcall jps
```

4.2.2 Hadoop 安装

1) 安装步骤

详见：尚硅谷大数据技术之 Hadoop（入门）



尚硅谷大数据技术
之Hadoop（入门）

2) 项目经验

（1）项目经验之 HDFS 存储多目录

① 生产环境服务器磁盘情况

```
[master@slave03 ~]$ df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda7        3.6T  1.6T  2.0T  45% /
devtmpfs         63G   0    63G   0% /dev
tmpfs            63G   20K   63G   1% /dev/shm
tmpfs            63G  714M   63G   2% /run
tmpfs            63G   0    63G   0% /sys/fs/cgroup
/dev/sdc1        954G  177G  777G  19% /hd3
/dev/sdd1        954G  177G  777G  19% /hd4
/dev/sdb1        3.7T  989G  2.7T  27% /hd2
/dev/sda5       1014M  123M  892M  13% /boot
```

② 在 hdfs-site.xml 文件中配置多目录，注意新挂载磁盘的访问权限问题。

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：尚硅谷官网

HDFS 的 DataNode 节点保存数据的路径由 `dfs.datanode.data.dir` 参数决定，其默认值为 `file://${hadoop.tmp.dir}/dfs/data`，若服务器有多个磁盘，必须对该参数进行修改。如服务器磁盘如上图所示，则该参数应修改为如下的值。

```
<property>
  <name>dfs.datanode.data.dir</name>

<value>file:///dfs/data1,file:///hd2/dfs/data2,file:///hd3/dfs/data3,file:///hd4/dfs/data4</value>
</property>
```

注意：每台服务器挂载的磁盘不一样，所以每个节点的多目录配置可以不一致。单独配置即可。

(2) 项目经验之集群数据均衡

① 节点间数据均衡

开启数据均衡命令：

```
start-balancer.sh -threshold 10
```

对于参数 10，代表的是集群中各个节点的磁盘空间利用率相差不超过 10%，可根据实际情况进行调整。

停止数据均衡命令：

```
stop-balancer.sh
```

② 磁盘间数据均衡

生成均衡计划（我们只有一块磁盘，不会生成计划）

```
hdfs diskbalancer -plan hadoop103
```

执行均衡计划。

```
hdfs diskbalancer -execute hadoop103.plan.json
```

查看当前均衡任务的执行情况。

```
hdfs diskbalancer -query hadoop103
```

取消均衡任务。

```
hdfs diskbalancer -cancel hadoop103.plan.json
```

(3) 项目经验之 Hadoop 参数调优

① HDFS 参数调优 `hdfs-site.xml`

The number of Namenode RPC server threads that listen to requests from clients. If `dfs.namenode.servicerpc-address` is not configured then Namenode RPC server threads listen to requests from all nodes.

NameNode 有一个工作线程池，用来处理不同 DataNode 的并发心跳以及客户端并发的元数据操作。

对于大集群或者有大量客户端的集群来说，通常需要增大参数 `dfs.namenode.handler.count` 的默认值 10。

```
<property>
  <name>dfs.namenode.handler.count</name>
```

```
<value>10</value>
</property>
```

$dfs.namenode.handler.count=20 \times \log_e^{Cluster Size}$, 比如集群规模为 8 台时, 此参数设置为 41。可通过简单的 python 代码计算该值, 代码如下。

```
[atguigu@hadoop102 ~]$ python
Python 2.7.5 (default, Apr 11 2018, 07:36:10)
[GCC 4.8.5 20150623 (Red Hat 4.8.5-28)] on linux2
Type "help", "copyright", "credits" or "license" for more information.
>>> import math
>>> print int(20*math.log(8))
41
>>> quit()
```

② YARN 参数调优 yarn-site.xml

情景描述: 总共 7 台机器, 每天几亿条数据, 数据源->Flume->Kafka->HDFS->Hive

面临的问题: 数据统计主要用 HiveSQL, 没有数据倾斜, 小文件已经做了合并处理, 开启的 JVM 重用, 而且 IO 没有阻塞, **内存用了不到 50%**。但是还是跑的非常慢, 而且数据量洪峰过来时, 整个集群都会宕掉。基于这种情况有没有优化方案。

解决办法:

内存利用率不够。这个一般是 Yarn 的 2 个配置造成的, 单个任务可以申请的最大内存大小, 和 Hadoop 单个节点可用内存大小。调节这两个参数能提高系统内存的利用率。

(a) yarn.nodemanager.resource.memory-mb

表示该节点上 YARN 可使用的物理内存总量, 默认是 8192 (MB), 注意, 如果你的节点内存资源不够 8GB, 则需要调减小这个值, 而 YARN 不会智能的探测节点的物理内存总量。

(b) yarn.scheduler.maximum-allocation-mb

单个任务可申请的最多物理内存量, 默认是 8192 (MB)。

4.2.3 Zookeeper 安装

1) 安装步骤

详见: 尚硅谷大数据技术之 Zookeeper



尚硅谷大数据技术
之 Zookeeper (V3)

4.2.4 Kafka 安装

1) 安装步骤

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载, 可百度访问: [尚硅谷官网](#)

详见：尚硅谷大数据技术之 Kafka



尚硅谷大数据技术
之Kafka (V3.0) .c

4.2.5 Flume 安装

按照采集通道规划，需在 hadoop102，hadoop104 两台节点分别部署一个 Flume。**可参照以下步骤先在 hadoop102 安装，然后再进行分发。**

1) 安装步骤

详见：尚硅谷大数据技术之 Flume



尚硅谷大数据技术
之Flume (V3.0) .c

2) 分发 Flume

```
[atguigu@hadoop102 ~]$ xsync /opt/module/flume/
```

3) 项目经验

（1）堆内存调整

Flume 堆内存通常设置为 4G 或更高，配置方式如下：

修改/opt/module/flume/conf/flume-env.sh 文件，配置如下参数（虚拟机环境暂不配置）

```
export          JAVA_OPTS="-Xms4096m                    -Xmx4096m  
-Dcom.sun.management.jmxremote"
```

注：

-Xms 表示 JVM Heap（堆内存）最小尺寸，初始分配。

-Xmx 表示 JVM Heap（堆内存）最大允许的尺寸，按需分配。

4.3 日志采集 Flume

4.3.1 日志采集 Flume 配置概述

按照规划，需要采集的用户行为日志文件存放在 hadoop102，故需要在该节点配置日志采集 Flume。日志采集 Flume 需要采集日志文件内容，并对日志格式（JSON）进行校验，然后将校验通过的日志发送到 Kafka。

此处可选择 TaildirSource 和 KafkaChannel，并配置日志校验拦截器。

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：尚硅谷官网

选择 TailDirSource 和 KafkaChannel 的原因如下：

1) TailDirSource

TailDirSource 相比 ExecSource、SpoolingDirectorySource 的优势。

TailDirSource：断点续传、多目录。Flume1.6 以前需要自己自定义 Source 记录每次读取文件位置，实现断点续传。

ExecSource 可以实时搜集数据，但是在 Flume 不运行或者 Shell 命令出错的情况下，数据将会丢失。

SpoolingDirectorySource 监控目录，支持断点续传。

2) KafkaChannel

采用 Kafka Channel，省去了 Sink，提高了效率。

日志采集 Flume 关键配置如下：



日志采集Flume关键配置



TailDirSource

```
#通过正则表达式匹配需要采集的日志文件
a1.sources.r1.filegroups=f1
a1.sources.r1.filegroups.f1=/opt/module/applog/log/app.*
```

KafkaChannel

```
#指定数据要发往的目标Topic
a1.channels.c1.kafka.topic = topic_log
```

让天下没有难学的技术

4.3.2 日志采集 Flume 配置实操

1) 创建 Flume 配置文件

在 hadoop102 节点的 Flume 的 job 目录下创建 file_to_kafka.conf。

```
[atguigu@hadoop102 flume-1.10.1]$ mkdir job
[atguigu@hadoop102 flume-1.10.1]$ vim job/file_to_kafka.conf
```

2) 配置文件内容如下

```
#定义组件
a1.sources = r1
a1.channels = c1

#配置 source
```

```
a1.sources.r1.type = TAILDIR
a1.sources.r1.filegroups = f1
a1.sources.r1.filegroups.f1 = /opt/module/applog/log/app.*
a1.sources.r1.positionFile =
/opt/module/flume-1.10.1/filetokafka/taildir_position.json

#配置 channel
a1.channels.c1.type = org.apache.flume.channel.kafka.KafkaChannel
a1.channels.c1.kafka.bootstrap.servers =
hadoop102:9092,hadoop103:9092
a1.channels.c1.kafka.topic = topic_log
a1.channels.c1.parseAsFlumeEvent = false

#组装
a1.sources.r1.channels = c1
```

4.3.3 日志采集 Flume 测试

1) 启动 Zookeeper、Kafka 集群

2) 启动 hadoop102 的日志采集 Flume

```
[atguigu@hadoop102 flume]$ bin/flume-ng agent -n a1 -c conf/ -f
job/file_to_kafka.conf
```

3) 启动一个 Kafka 的 Console-Consumer

```
[atguigu@hadoop102 kafka]$
```

4) 生成数据

执行集群日志生成脚本。

```
[atguigu@hadoop102 ~]$ lg.sh test 100
```

5) 观察 Kafka 消费者是否能消费到数据

4.3.4 日志采集 Flume 启停脚本

1) 在 hadoop102 节点的/home/atguigu/bin 目录下创建脚本 fl.sh

```
[atguigu@hadoop102 bin]$ vim fl.sh
```

在脚本中填写如下内容。

```
#!/bin/bash

case $1 in
"start"){
    echo " -----启动 hadoop102 采集 flume-----"
    ssh hadoop102 "nohup /opt/module/flume-1.10.1/bin/flume-ng agent
-n a1 -c /opt/module/flume-1.10.1/conf/ -f
/opt/module/flume-1.10.1/job/file_to_kafka.conf >/dev/null 2>&1 &"
};;
"stop"){
    echo " -----停止 hadoop102 采集 flume-----"
    ssh hadoop102 "ps -ef | grep file_to_kafka | grep -v grep |awk
'{print \$2}' | xargs -n1 kill -9 "
};;
esac
```

2) 增加脚本执行权限

```
[atguigu@hadoop102 bin]$ chmod 777 fl.sh
```

3) fl 启动

```
[atguigu@hadoop102 module]$ fl.sh start
```

4) fl 停止

```
[atguigu@hadoop102 module]$ fl.sh stop
```